DIAGRAMME DE CLASSES

INTRODUCTION

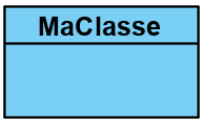
Un diagramme de classes est une représentation graphique des classes d'un système. Il permet d'un rapide coup d'œil de comprendre la structure générale du système. On y voit toutes les classes d'un programme et leurs relations, sans devoir examiner une grande quantité de fichiers de code source.

Afin de standardiser les diagrammes, une norme graphique a été inventée, avec une sémantique particulière. Un peu comme un langage de programmation. L'**UML** (acronyme anglais de « *Unified Modeling Language* », soit le « Langage de Modélisation Unifié ») est une norme graphique très générale qui permet de créer plusieurs types de modèles. Les diagrammes de classes sont un type parmi tous ceux fournis par la norme.

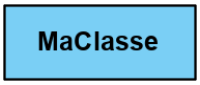
REPRÉSENTATION D'UNE CLASSE

Dans un diagramme de classe UML, une classe est représentée par un rectangle. Celui-ci peut prendre 2 formes :

Forme standard



Forme simplifiée



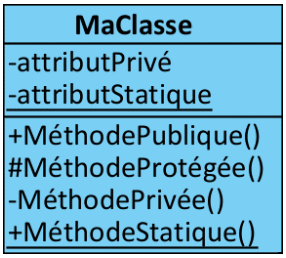
Il est très important que le nom de la classe dans le diagramme **soit identique** au nom de la classe dans le code.

Classe abstraite

Dans le cas d'une classe abstraite, le nom de la classe est écrit en italique.

Forme complète

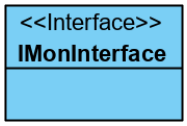
La norme UML permet de définir complètement une classe dans un diagramme, c'est-à-dire d'y inclure toute l'information présente dans le code. La classe est alors représentée dans un rectangle divisée en 3 sections : le nom en haut, les attributs au milieu et les méthodes en bas.



Cette notation n'est utilisée qu'avec des outils qui permettent une traduction automatique du code en diagramme et vice-versa.

Dans l'utilisation courante, **cette forme est à proscrire**. Elle nécessite de maintenir à jour une grande quantité d'information à 2 endroits différents. Les risques d'avoir une divergence entre les deux sont quasi assurés. Le but d'un diagramme de classe est de donner une **vue d'ensemble** sur toute les classes et la façon dont elles interagissent, ce qui est difficile à voir sans parcourir un grand nombre de fichiers de code source. Les détails d'une classe sont facilement accessibles dans un fichier de code.

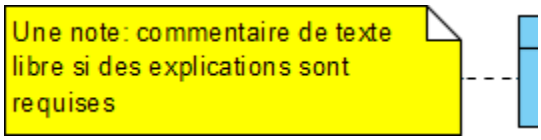
Interface



Dans un diagramme de classe UML, il n'y a pas de symbole différent pour représenter une interface. Une interface est représentée comme une classe. Mais comme il y a une différence dans le code, il y a une différence dans le diagramme.

Au-dessus du nom de la classe est ajoutée une indication **<<Interface>>**. Les doubles crochets << et >> de chaque côté son important pour respecter la syntaxe.

Note

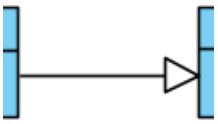


Un rectangle dont le coin supérieur droit est replié vers l'intérieur correspond à une note. Ceci ne représente pas un élément du code, mais un commentaire explicatif. La note peut être indépendante, ou attachée à un élément.

Relations

Dans un programme, les classes ne sont pas des éléments indépendants, existants seuls. Elles sont reliées entre elles par différents types de relations. Chaque type de relation a sa représentation graphique en UML.

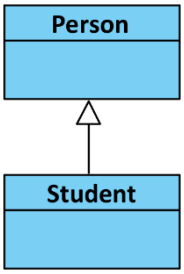
Héritage



Une relation d'héritage entre 2 classes est représentée par un trait reliant les deux classes et dont l'extrémité du côté de la classe mère comporte un triangle. Il s'agit donc d'une flèche qui pointe vers la classe de base.

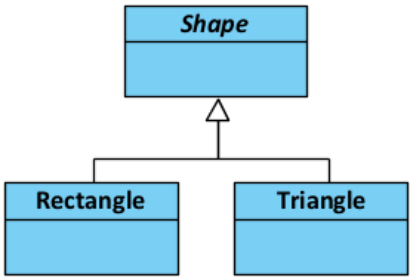
Ainsi le code suivant est représenté graphiquement par ce diagramme.

```
class Person
{
}

class Student : Person
{
}
```



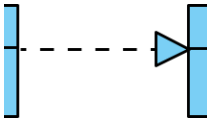
Ou encore

```
abstract class Shape
{
}

class Rectangle : Shape
{
}

class Triangle : Shape
{
}
```



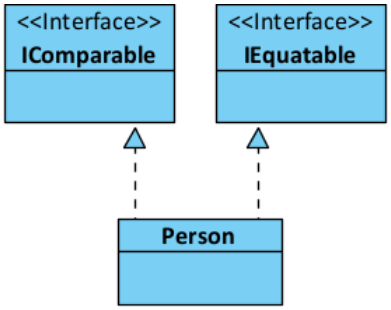
Implémentation

En UML, une implémentation est une relation très similaire à un héritage, mais est utilisée avec les interfaces. La représentation graphique est très similaire à un héritage, mais la ligne reliant la classe à l'interface est discontinue.



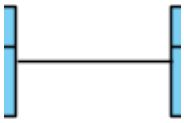
Une classe peut implémenter plus d'une interface, il est donc possible d'avoir plusieurs relations d'implémentation pour une même classe.

```
class Person :
   IEquatable<Person>, IComparable<Person>
{
}
```



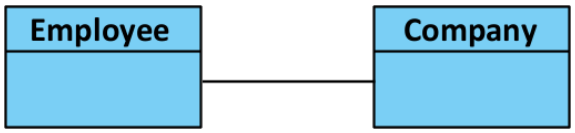
Association

Une association est une relation qui existe entre 2 classes lorsqu'un objet d'une classe est un attribut d'une autre classe. En UML, une association est représentée par une simple ligne pleine. Il est par contre possible d'ajouter de l'information pour décrire l'association. Il est de plus possible de terminer une extrémité ou l'autre de la ligne avec un symbole pour ajouter une sémantique à la relation.

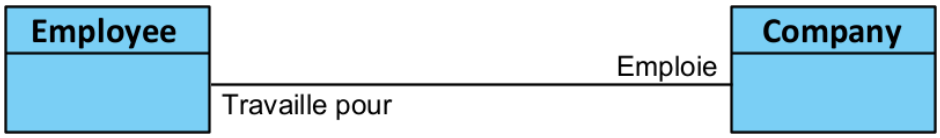


Le code suivant est représenté graphiquement par ce diagramme.

```
class Employee
{
   private Company _employer;
}

class Company
{
   private List<Employee> _employees;
}
```



Si ce n'est pas évident à la lecture du diagramme, il est possible d'ajouter une information à l'extrémité d'une association pour spécifier le rôle de la classe dans la relation.



- L'employé travaille pour la compagnie
- La compagnie emploie l'employé

L'autre façon d'ajouter de l'information à une association est en lui donnant un nom. Avec ce nom peut être ajoutée une flèche indiquant le sens de la relation.

⌘ La compagnie est l'employeur de l'employé

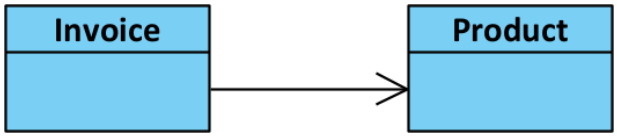
NAVIGABILITÉ

Dans l'exemple précédent, la classe « **`Employee`** » a un attribut de type « **`Company`** », et la classe « **`Company`** » a un attribut de type « **`Employee`** » (plus précisément une liste d'employés). L'employé connait son employeur, et la compagnie connait ses employés. La relation est bidirectionnelle. C'est pourquoi dans le diagramme il n'y a pas de flèche aux extrémités de la ligne.

Dans un système de gestion d'inventaire d'un magasin se retrouvent les classes « **`Product`** » et « **`Invoice`** ».

```
class Product
{
}

class Invoice
{
   private List<Product> _items;
}
```

Lorsqu'un client achète des produits, ils sont listés sur la facture. La facture a donc une liste de produits. Par contre, un produit ne connait pas les factures sur lesquelles il est listé. Il n'y a pas de façon de naviguer d'un produit jusqu'à une facture. L'association est unidirectionnelle. Ceci se représentera dans le diagramme par une flèche à une extrémité de la ligne.



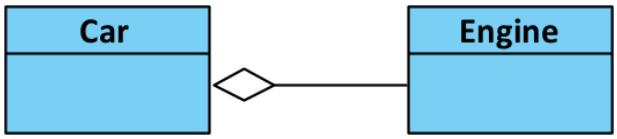
⌘ Une facture connait ses produits. Un produit ne connait pas ses factures.

AGRÉGATION

L'agrégation est une variation sémantique de la simple association. Elle représente une relation du type « *possède* » ou « *est composé de* », lorsqu'un objet est une partie d'un plus gros objet.

Un ordinateur est composé d'une carte mère, d'un processeur, d'un disque. Un camion de pompier possède une échelle. Une bibliothèque possède des livres.

Pour représenter une agrégation dans un diagramme UML, un losange vide est ajouté à l'extrémité de l'association du côté du contenant, la classe qui possède l'autre. Une voiture est composée d'un moteur :



L'agrégation est une notion purement sémantique. Dans le code, il n'y a pas de différence entre une agrégation et une simple association. Dans certains cas, il est même difficile de répondre à la question : « *Est-ce une agrégation ou une simple association ?* »

Bien qu'il soit important de connaitre la signification de l'agrégation pour comprendre un diagramme lors de sa lecture, il est correct de ne pas l'utiliser lors de la création d'un diagramme et de la considérer comme une association.

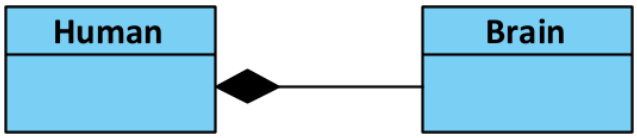
Les auteurs de la norme UML eux-mêmes indiquent que « ***Si vous ne comprenez pas bien l'agrégation, ne l'utilisez pas.*** »

Composition

La composition est aussi une variation sémantique de la simple association. Comme l'agrégation, elle représente une relation du type « *possède* » ou « *est composé de* », lorsqu'un objet est une partie d'un plus gros objet.

Mais contrairement à l'agrégation, la composition a **des répercussions dans le code**. Lorsqu'un objet est composé d'un autre, cela signifie que si l'objet englobant cesse d'exister, l'objet englobé cesse lui aussi d'exister.

Par exemple :



Un humain possède un cerveau. Par contre, le cerveau n'existe pas sans humain. Lorsque l'humain meurt, le cerveau meurt avec.

Par opposition, une voiture possède un moteur, mais il est possible de récupérer le moteur et le transférer à une autre voiture si la voiture est détruite.

La voiture agrège un moteur. L'humain est composé d'un cerveau.

Pour représenter une composition dans un diagramme UML, un losange plein est ajouté à l'extrémité de l'association du côté du contenant, la classe qui possède l'autre.

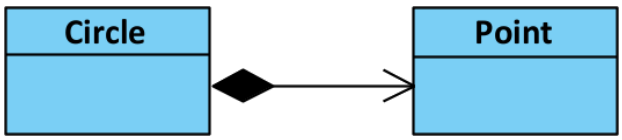
La composition est importante pour comprendre la durée de vie des objets dans le programme, et savoir à quel moment un objet peut être utilisé, et à partir de quel moment il ne peut plus l'être. Dans certains langages comme le *C++*, ceci aura un impact sur le type de variable utilisé.

Navigabilité

Comme pour la simple association, il est possible de spécifier la navigabilité entre les classes dans une composition. Il va sans dire que la classe composée connait toujours la classe composant. Dans l'exemple précédent, aucune flèche n'est indiquée. Ceci indique qu'il est possible de naviguer du cerveau vers l'humain.

Dans certains cas, la composition est unidirectionnelle. L'exemple suivant présente une classe « `Circle` » qui a un attribut de type « `Point` » pour représenter la position de son centre. Le cercle est composé d'un point, car si le cercle cesse d'exister, son centre cesse d'exister en même temps. Par contre le point ne sait pas qu'il fait partie d'un cercle.

```
class Point
{
}

class Circle
{
   private Point _center;
}
```



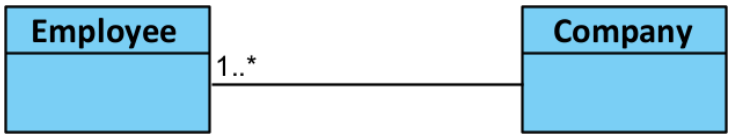
Cardinalité

Dans une association, la cardinalité exprime le nombre d'**objets** qui peuvent exister dans le programme. Il s'agit donc d'un renseignement sur le comportement du programme lors de son exécution.

La cardinalité peut s'exprimer de différentes façons :

- Un nombre fixe : 1, 2, 10, etc. (1 sera souvent omis)
- Un intervalle `M..N`, qui signifie minimum M occurrences, maximum N occurrences. Par exemple :
  - `0..1` : Zéro ou un
  - `2..4` : Entre deux et quatre
  - `1..*` : Un ou plusieurs. Le symbole '`*`' représente un nombre indéterminé, ou infini.
  - `*` : Zéro ou plusieurs (version abrégée '`0..*`')

Par exemple :



La cardinalité s'applique à la classe la plus proche. Le diagramme ci-dessus se lit :

- La compagnie fait travailler 1 à plusieurs employés. Si elle n'a pas d'employé, elle n'existe pas.
- L'employé travaille pour 1 compagnie

Une ou plusieurs compagnies ?

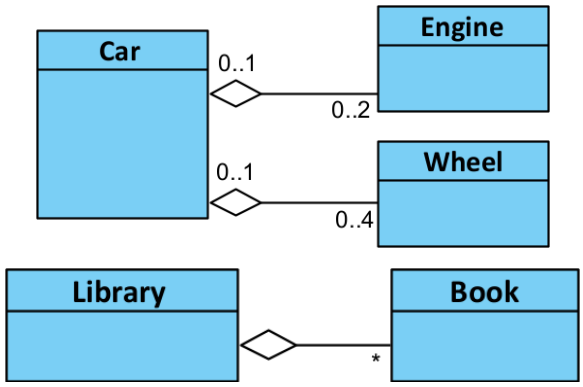
Est-ce que la cardinalité de la compagnie devrait être `1` ou `*` ? Théoriquement, une personne peut travailler pour plusieurs compagnies.

Un programme informatique n'est pas un système théorique. Il existe pour accomplir une tâche et répondre à un besoin. Le modèle doit s'appliquer aux besoins du programme.

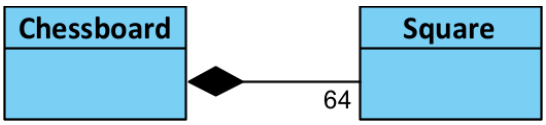
Dans un système de gestion d'employés d'une compagnie, la cardinalité de la compagnie sera 1. Même si certains employés travaillent aussi pour d'autres compagnies, le programme n'a pas à gérer cette information.

Dans un système pour le service des impôts, il faut tenir compte de tous les emplois d'un travailleur. La cardinalité de la compagnie sera `*`.

Autres exemples :



Lors d'une composition, la cardinalité du côté de la classe composée sera toujours 1.



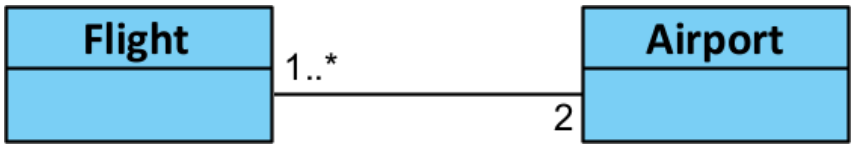
- Un échiquier est composé de 64 cases
- Une case appartient à un et un seul échiquier

Associations distinctes

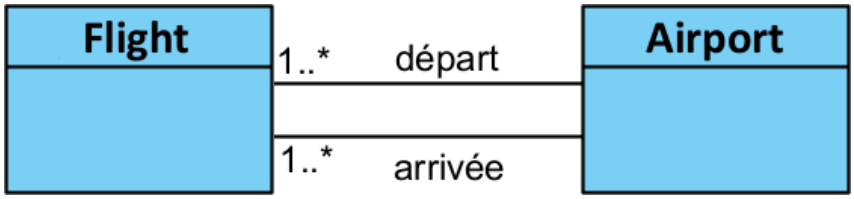
Dans certains cas, différentes associations existent entre 2 classes. Par exemple, un vol d'avion est associé à 2 aéroports, le point de départ et le point d'arrivée.

```
class Flight
{
   private Airport _departure;
   private Airport _arrival;
}
```

Il est possible de représenter l'association avec une cardinalité.



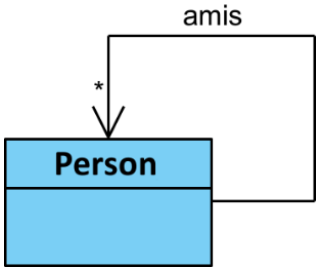
Mais les 2 aéroports ne sont pas génériques, elles ont une signification particulière. Dans ce cas, il est préférable de représenter 2 associations et d'ajouter un nom ou un rôle pour clairement indiquer la signification de l'association.



AUTOASSOCIATION

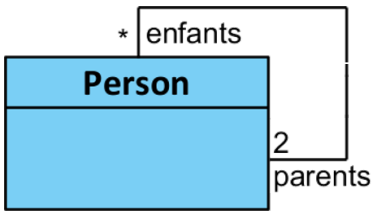
Une autoassociation est une association vers soi-même. Cela se produit lorsqu'une classe a un attribut de son propre type. Par exemple, une personne peut avoir des amis, qui sont eux-mêmes des personnes.

```
class Person
{
   private List<Person> _friends;
}
```



Un cas similaire, mais un peu plus complexe. Une personne a une mère et un père, et peut avoir des enfants. La classe « **Person** » serait alors définie ainsi :

```
class Person
{
   private Person _mother;
   private Person _father;
   private List<Person> _children;
}
```



DÉPENDANCE

Une relation de dépendance existe entre 2 classes lorsqu'une d'entre elles utilise l'autre, soit comme

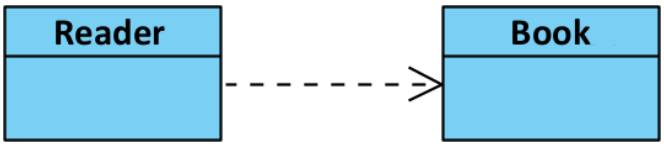
- variable locale dans une méthode,
- paramètre à une méthode,
- type de retour d'une méthode.

Une relation de dépendance existe seulement si une association n'existe pas. Si une association existe, la dépendance n'est pas indiquée dans le diagramme.

Dans l'exemple suivant, la classe « **Reader** » reçoit un paramètre de type « **Book** » dans sa méthode « **Read** ».

```
class Reader
{
   public void Read(Book book)
   {
      // Lire le livre donné
   }
}
```

Dans un diagramme de classe UML, une dépendance est représentée par une flèche formée d'une ligne discontinue pointant vers la classe utilisée.



⌘ Le lecteur dépend du livre.